IACR Communications in Cryptology
ISSN 3006-5496, Vol. 2, No. 4, 23 pages.

https://doi.org/10.62056/avom-4tw9

Check for updates

Running Standard Block Ciphers Beyond AES with TFHE: Experiments and Lessons Learnt

Daphné Trama, Aymen Boudguiga and Renaud Sirdey

Université Paris-Saclay, CEA-List, Palaiseau, France

Abstract. The dream of achieving data privacy during external computations has become increasingly concrete in recent years. Indeed, since the early days of Fully Homomorphic Encryption (FHE) more than a decade ago, new cryptosystems and techniques have constantly optimized the efficiency of computation on encrypted data. However, one of the main disadvantages of FHE, namely its significant ciphertext expansion factor, remains at the center of the efficiency bottleneck of FHE schemes. To tackle the issue of slow uplink FHE data transmission, we use transciphering. With transciphering, the client naturally encrypts its data under a symmetric scheme and sends them to the server with (once and for all) an FHE encryption of the symmetric scheme's key. With its larger computing power, the server then evaluates the symmetric scheme's decryption algorithm within the homomorphic domain to obtain homomorphic ciphertexts that allow it to perform the requested calculations. Since the first use of this method a bit more than ten years ago, papers on the homomorphic evaluation of AES have been numerous. And as the AES execution is the application chosen by NIST in the FHE part of its recent call for proposals on threshold encryption[1], the stakes of such work go up another level. But what about other standardized block ciphers? Is the AES the more efficient option? In this work, we leverage on two methods which have successfully been applied to the homomorphic evaluation of AES [TCBS23, BBB+25b] to study several state-of-the-art symmetric block ciphers (namely CLEFIA, PRESENT, PRINCE, SIMON, SKINNY). That is to say, we implement a representative set of symmetric block ciphers using TFHE. These implementations allow us to compare the efficiency of this set of symmetric schemes and to categorize them. We highlight the characteristics of block ciphers that are fast to execute in the homomorphic domain and those that are particularly costly. Finally, this classification of operation types enables us to sketch out what the ideal block cipher for transciphering homomorphic data in integer mode might look like.

Keywords: Symmetric Block Cipher · Homomorphic Encryption · Transciphering · TFHE

1 Introduction

The first attempt to transcipher AES ciphertexts into FHE data dates back to 2012 with the work of Gentry, Halevi, and Smart [GHS12]. Their implementation of AES with BGV under HElib ran with an execution latency of 17.5 minutes. Their work thus showed that the approach could not be used in practice to solve the issue of slow data transmission, or at least not with AES. However, transciphering, which refers to the ability to convert low-overhead symmetrically encrypted data into homomorphically encrypted data, remains a

This work was supported by the France 2030 ANR Project ANR-22-PECY-003 SecureCompute.
E-mail: aymen.boudguiga@cea.fr (Aymen Boudguiga), renaud.sirdey@cea.fr (Renaud Sirdey)
[1] https://csrc.nist.gov/Projects/threshold-cryptography.

This work is licensed under a "CC BY 4.0" license.
Received: 2025-10-06 Accepted: 2025-12-02

critical issue for the practicality of FHE. As a result, several teams have focused on creating new symmetric cryptosystems designed specifically to accelerate encryption operations within the homomorphic domain. Currently, there are numerous proposals, including block ciphers such as LowMC [ARS+15], PRINCE [BCG+12], and CHAGHRI [AMT22], as well as stream ciphers like Elisabeth [CHMS22], PASTA [DGH+23], and Kreyvium [CCF+16]. This group of new schemes, known as hybrid encryption schemes, propose several fast and efficient homomorphic executions compared to [GHS12], but none have been standardized. In 2022, the NIST announced a future call for treshold encryption with a specific FHE part. NIST also explained that AES would be the application chosen as a benchmark for FHE propositions. That is why, since 2022, many teams have been reworking AES transciphering. Since the breakthrough of [TCBS23] in 2023, which brought execution times under 5 minutes in sequential mode and 30 seconds in parallelized mode, other teams have been bringing down execution times in sequential mode. All these works have one thing in common: they use TFHE for their homomorphic evaluation of AES. Indeed, block ciphers are inherently designed around the iteration of a non-linear function over several rounds. For this reason, they inherently induce pretty large multiplicative depths which make them unsuited to levelled/somewhat schemes. Because of TFHE boostrapping ability, it is not sensitive to this multiplicative-depth issue and, as such, is the most suited FHE to be able to run this kind of algorithms. But this aforementionned TFHE common denominator is where the similarity ends, as there are almost as many AES-over-TFHE records as there are methods: [TCBS23] uses TFHE's Programmable Bootstrapping in integer mode, and especially a method called Tree-Based Method (TBM) [GBA21] that allows bootstrapping on several encrypted inputs. Then, at the end of 2023 [BPR24] proposed a new method for working with binary ciphertexts in TFHE, the *p-encodings* method, and managed to evaluate the AES in 211 seconds. Several works by other teams (Fregata [WWL+23], Thunderbird [WLW+24]) use TFHE in LHE mode and leverage on circuit bootstrapping (achieving 86 seconds, then 46 seconds on a single core) or [SAP24] which relies on an original bit-oriented LUT evaluation (then achieving a single-core timing of 67 seconds). Finally, in 2025, a new framework called Hippogryph [BBB+25b] obtained the best execution timings by leveraging the integer approach of [TCBS23] for the Sbox evaluation and the *p-encodings* method for the others steps. AES is certainly an interesting benchmark for measuring technical advances in FHE, but what about other standardized block ciphers? In this article, we take a look at these less-studied ciphers (from an FHE perspective). Note that there already are numerous works on stream ciphers [MJSC16, CCF+16, BBS21, BOS23, BCBS23, BBB+25a], contrary to the case of block ciphers. This state of affairs is explained by two main reasons. First, the study of transciphering was ignited before the first FHE with "efficient" bootstrapping existed, so multiplicative depth was an issue and genuine stream ciphers where shown to induce lower multiplicative depths. Second, stream ciphers have very desirable properties for transciphering. Indeed, most of the computational burden of transferring yields virtually no latency since the homomorphic keystream can be computed offline.For example, when the keystream takes its values directly in $\mathbb{Z}_t$, where t is the plaintext modulus, transciphering itself is only a mere addition[2]. Even if transciphering with symmetric block ciphers is thus usually less efficient than with stream ciphers, it remains interesting to work on them. Also because block ciphers are more versatile cryptographic primitives, for example allowing to build FHE-friendly hash functions which can be useful in some cases ([BSS+23]). Moreover, the study of block ciphers for transciphering is not without interest, as a recent survey on the subject reminds us [TKLP25].

[2]This operation can still be done efficiently, for example when the client uses a stream cipher for $\{0,1\}$ while the server works with an arbitrary plaintext domain $\mathbb{Z}_t$, $t > 2$, using the plaintext switching techniques developped in [CDPP22, MPP24]

Contributions In this work, we attempt to execute several state-of-the-art standardized symmetric block ciphers[3] (namely AES, PRINCE, PRESENT, SKINNY, CLEFIA, SIMON) over TFHE ciphertexts, by means of TFHE Programmable Bootstrapping. To do so, leveraging on the method introduced in [TCBS23], we first transform all the algorithms into a sequence of LookUp Table (LUT) evaluations for 8-bit inputs decomposed into two nibbles. Then, using home-made optimized TFHE operators, we implement these symmetric schemes with the TFHElib library. While this approach may not be the fastest in the state-of-the-art, it is a useful metric for comparing different schemes. Building on the results we obtain, we then leverage on the Hippogryph framework [BBB+25b] for a second analysis. This second study allows us to determine whether the conclusions drawn from the first approach are consistent or if new and interesting insights emerge from the second framework. One of our finding, which is counter-intuitive since AES seemed to have the reputation of not being "FHE-friendly", is that it comes only as a close second to the PRINCE lightweight block cipher. As a bonus contribution, we propose a method based on the number of BlindoRotate and TLWE-to-TRLWE Keyswitch to estimate execution timings. In addition, we propose a classification of these symmetric block ciphers according to their compatibility with FHE evaluation. Indeed, we identify several categories of operations that determine the efficiency of the different schemes in the homomorphic domain. Finally, we sketch an ideal (yet hypothetical) block cipher based on the criteria established in our study.

2 Preliminaries

2.1 Notations

Unless otherwise stated, the (uppercase or lowercase) letter c always denotes a ciphertext. Other (uppercase or lowercase) letters denote plaintexts.

Let $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ be the real torus, that is to say, the additive group of real numbers modulo 1 ($\mathbb{R} \mod 1$). We further denote by $\mathbb{T}_N[X]^n$ the set of vectors of size n whose coefficients are polynomials of $\mathbb{T}[X] \mod (X^N + 1)$. N is usually a power of 2. Let $\mathbb{B} = \{0, 1\}$. $\langle\ ,\ \rangle$ denotes the inner product.

2.2 The TFHE Scheme

The TFHE scheme is a fully homomorphic encryption scheme [CGGI16] notably implemented in the TFHE library[4]. TFHE defines three structures to encrypt plaintexts, which we introduce below:

- TLWE sample: A pair $(a, b) \in \mathbb{T}^{n+1}$, where a is uniformly sampled from $\mathbb{T}^n$ and $b = \langle a, s\rangle + e$. The secret key s is uniformly sampled from $\mathbb{B}^n$, and the error $e \in \mathbb{T}$ is sampled from a Gaussian distribution with mean 0 and standard deviation σ.
- TRLWE sample: A pair $(a, b) \in \mathbb{T}_N[X]^{k+1}$, where a is uniformly sampled from $\mathbb{T}_N[X]^k$ and $b = \langle a, s\rangle + e$. The secret key s is uniformly sampled from $\mathbb{B}_N[X]^k$, the error $e \in \mathbb{T}_{\mathbb{N}}[\mathbb{X}]$ is a polynomial with random coefficients sampled from a Gaussian distribution with mean 0 and standard deviation σ. One usually chooses $k = 1$; therefore, a and b are polynomials.
- TRGSW sample: a vector of $(k + 1)l$ TRLWE fresh samples.

[3]The main motivation behind the choice of these algorithms is to be included in some standard. This may be a little bit arbitrary but we consider this a synonym of well-respected algorithms that have stand the test of time. This is to the exception of PRINCE which is, to the best of our knowledge, not included in any standard.

[4]tfhe.github.io/tfhe/

Let $\mathcal{M}$ denote the discrete message space ($\mathcal{M} \in \mathbb{T}_N[X]$ or $\mathcal{M} \in \mathbb{T}$)[5]. To encrypt a message $m \in \mathcal{M}$, we add what is called a *noiseless trivial* ciphertext $(0, m)$ to a fresh encryption of 0. We denote by $c = (a, b) + (0, m) = (a, b + m) \in \mathrm{T(R)LWE}_s(m)$ the T(R)LWE encryption of m with key s. A message $m \in \mathbb{Z}[X]$ can also be encrypted in TRGSW samples by adding $m \cdot H$ to a TRGSW sample of 0, where H is a gadget decomposition matrix. As we will not explicitly need such an operation in this paper, more details about TRGSW can be found in [CGGI16]. To decrypt a ciphertext c, we first calculate its phase $\phi(c) = b - \langle a, s \rangle = m + e$. Then, we need to remove the error, which is achieved by rounding the phase to the nearest valid value in $\mathcal{M}$. This procedure fails if the error exceeds half the distance between two consecutive elements of $\mathcal{M}$.

2.3 Bootstrapping and Programmable Bootstrapping

2.3.1 TFHE bootstrapping.

Bootstrapping is the operation that reduces the noise of a ciphertext, enabling further homomorphic calculations. It relies on three basic operations, which we briefly review in this section. The first operation, BlindRotate (BR), rotates a plaintext polynomial *testv* (sometimes referred to as the test polynomial or vector) by a TLWE-encrypted index $c \in [\![m]\!]$. It returns a TRLWE-encrypted polynomial of $testv \cdot X^{\phi(c)} \mod (X^N + 1)$, where $\phi(c)$ is the phase of c rescaled in $\mathbb{Z}_{2N}$. Then, one must apply a TLWESampleExtract, which extracts a coefficient from an encrypted TRLWE polynomial and converts it into the corresponding TLWE ciphertext. Finally, the PublicFunctionalKeyswitch enables the switching of ciphertext formats. It is used to switch the extracted TLWE ciphertext to an encryption of the same message but with the initial key. In practice, the computation time for a bootstrapping mainly depends on the efficiency of the BlindRotate [CBSZ23]. *From now on, we will denote by* $\mathsf{N_{BR}}$ *the number of* BlindRotate *operations required to evaluate a function on encrypted data.*

2.3.2 Programmable bootstrapping.

Bootstrapping involves performing an indirection in a table using an encrypted index, which returns a ciphertext corresponding to the value in the table at the encrypted index with a smaller noise than the one in the index encryption. As any computation has the potential to be converted into a Lookup Table (LUT), we can carefully assign the LUT coefficients to the test polynomial. With such a test polynomial, bootstrapping does not only produce a new ciphertext with reduced noise, but a ciphertext on which a function was evaluated. Indeed, if we assign the coefficients of *testv* to the results of evaluating a function f on elements of $\mathcal{M}$, then applying bootstrapping to this new *testv* will yield $c' \in [\![f(m)]\!]$. In other words, bootstrapping produces an encryption of $f(m)$ without incurring any additional cost, enabling the evaluation of a LUT for f for free. This type of bootstrapping is referred to as *programmable* or *functional*. It is important to note that the original bootstrapping method described in [CGGI16] is a specific case of programmable bootstrapping, where the function f is set to the identity function.

3 Method and Tools

To compare the efficiency of different standardized block ciphers, we propose using the same methods and tools as outlined in [TCBS23], which presents a homomorphic AES evaluation based on LUT evaluation through TFHE functional bootstrapping. The tools

[5] In practice, we discretize the torus with respect to our plaintext modulus. For example, if we want to encrypt $m \in \mathbb{Z}_4 = \{0, 1, 2, 3\}$, we encode it in $\mathbb{T}$ as a value in $\mathcal{M} = \{0, 0.25, 0.5, 0.75\}$. As a slight abuse of notation, we use $\mathcal{M}$ to denote both the message space and its encodings on $\mathbb{T}$.

and techniques discussed in this paper have been further explored and optimized in [TCB+25], where a general-purpose TFHE processor abstraction was implemented. In our study, we utilize the optimized tools from this later work. This section first presents a brief overview of the tools and results proposed in [TCBS23] and [TCB+25]. Following this, we explain how to construct a generic method based on [TCBS23] for the homomorphic evaluation of any block cipher. It is important to note that while this approach may not be the fastest in the state of the art, it serves as a helpful metric for comparing different schemes. Additionally, we then conduct a second study on the same block ciphers using a different framework, namely Hippogryph [BBB+25b]. The Hippogryph framework is a recent work that proposes a new approach for transciphering (that can be seen as an optimization to [TCBS23]) and applies it to the AES, bringing the latency evaluation down to 32 seconds on a single core. This second study allows us to determine whether the conclusions drawn from the first approach are consistent or if new and interesting insights emerge from the second framework.

3.1 The full-LUT Approach

Building on the study presented in [TCBS23], which has been validated and explored in greater detail in [TCB+25], we chose to work with 8-bit plaintexts, which are divided into two 4-bit digits known as nibbles. The approach outlined in [TCBS23] involves transforming all AES steps into LUT evaluations, including XOR operations and multiplication in $GF(256)$. The authors also conduct a study to identify the most effective bootstrapping method and the optimal set of parameters (to use with TFHElib library). They concluded that combining the Tree-Based Method [GBA21] and the Multi-Value Bootstrapping [CIM19] was the most efficient approach. Thus, we decided to use their parameters and methods to compare AES with other block ciphers. For completion, we give in Appendix (Table 2) the concrete set of parameters from [TCB+25] that we used in our work.

3.1.1 Tree-Based Method.

Almost all state-of-the-art functional bootstrapping methods ([CJP21, KS25, YXS+21, CLOT21, CBSZ23]) take as input a single ciphertext from a relatively small set of messages. In [TCBS23], the authors rely on the *Tree-Based Method* (TBM), introduced in 2021 by Guimarães *et al.* [GBA21]. This method enables bootstrapping on multiple encrypted inputs, making it particularly suitable for evaluating multivariate functions. For example, it is used in [TCBS23] to evaluate XOR operations on 4-bit payload ciphertexts.

Figure 1 illustrates the XOR example applied to these 4-bit payload ciphertexts. We adopt this approach (that we named "the full-LUT methodology") for other block ciphers that also necessitate XOR evaluations.

The TBM works the following way: let $m_0, \cdots, m_{d-1}$ be d messages in $[\![0, B-1[\![$. From these messages, we obtain d TLWE encryptions $(c_0, c_1, \cdots, c_{d-1})$ of $(m_0, m_1, \cdots, m_{d-1})$ on half of the torus $\mathbb{T}$. We denote $f : [\![0, B-1]\!]^d \to [\![0, B'-1]\!]$ the target function and define g as the following bijection:

$$\begin{array}{rccc} g & : & [\![0, B-1]\!]^d & \to & [\![0, B^d-1]\!] \\ & & (a_0, a_1, \cdots, a_{d-1}) & \mapsto & \sum_{i=0}^{d-1} a_i \cdot B^i \end{array}$$

We then encode a LUT for f under B^{d-1} trlwe ciphertexts. Each of these ciphertexts encodes a polynomial P_i such that:

$$P_i(X) = \sum_{j=0}^{B-1} \sum_{k=0}^{\frac{N}{B}-1} f \circ g^{-1}(j \cdot B^{d-1} + i) \cdot X^{j \cdot \frac{N}{B} + k}$$

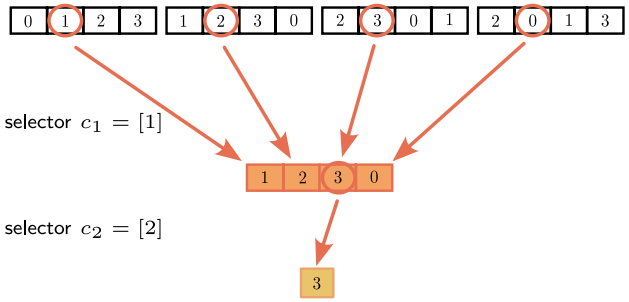


Figure 1: Illustration of the TBM on the LUT of the XOR with messages in $\{0, 1, 2, 3\}$. The messages are $m_1 = 1$ and $m_2 = 2$ and their corresponding encryptions are c_1 and c_2. Orange arrows indicate bootstrapping. Figure inspired from [TCBS23].

Then, we apply the BlindRotateAndExtract (the BlindRotate directly followed by the SampleExtract in position 0) to each test polynomial $testv = \mathsf{TRLWE}(P_i)$ with c_0 as a selector. We obtain B^{d-1} tlwe ciphertexts, each corresponding to the encryption of $f \circ g^{-1}(m_{d-1} \cdot B^{d-1} + i)$, for $i \in [\![0, B^{d-1} - 1]\!]$.

Finally, we use a TLWE-to-TRLWE KeySwitch to gather them into B^{d-2} encrypted TLWE, corresponding to the LUT of h, with:

$$\begin{array}{rccc} h & : & [\![0, B-1]\!]^{d-1} & \to & [\![0, B'-1]\!] \\ & & (a_0, a_1, \cdots, a_{d-2}) & \mapsto & f(a_0, a_1, \cdots, a_{d-2}, m_{d-1}) \end{array}$$

We then repeat this operation, using the ciphertext c_i at step i, until we obtain a single TLWE ciphertext of $f(m_0, m_1, \cdots, m_{d-1})$.

Note that in practice, we implement two different PublicFunctionalKeyswitch methods. The first allows multiple TLWE ciphertexts to be packed into a single TRLWE ciphertext, while the second switches the keys of a TLWE sample. The first key switch significantly impacts the computation time of tree-based functional bootstrapping, taking approximately $2\times$ longer than a BlindRotate. Therefore, we will refer to the number of calls to PublicFunctionalKeyswitch (KS) for packing TLWE ciphertexts into one TRLWE as $\mathsf{N_{KS}}$.

For the specific example of XOR, the TBM requires 5 BR operations ($\mathsf{N_{BR}}= 5$) and 1 KS operation ($\mathsf{N_{KS}}= 1$).

3.1.2 Multi-Value Bootstrapping.

Multi-Value Bootstrapping (MVB) [CIM19] refers to the method for evaluating k different LUTs on a single input with a single BlindRotate. As the blind rotation is the most expensive operation of the bootstrapping, this can easily lead to interesting execution timings. The key idea of MVB is to factor the test polynomial P_{f_i} associated with the function f_i into a product of two polynomials v_0 and v_i, where v_0 is a common factor to all P_{f_i}. In practice, we have:

$$(1 + X + \cdots + X^{N-1}) \cdot (1 - X) \equiv 2 \bmod (X^N + 1)$$

Now by writing P_{f_i} in the form $P_{f_i} = \sum_{j=0}^{N-1} \alpha_{i,j} X^j$ with $\alpha_{i,j} \in \mathbb{Z}$, we get from the previous equation:

$$\begin{aligned} P_{f_i} &= \frac{1}{2} \cdot (1 + X + \cdots + X^{N-1}) \cdot (1 - X) \cdot P_{f_i} \bmod (X^N + 1) \\ &= v_0 \cdot v_i \bmod (X^N + 1) \end{aligned}$$

where:

$$v_0 = \frac{1}{2} \cdot (1 + X + \cdots + X^{N-1})$$
$$v_i = \alpha_{i,0} + \alpha_{i,N-1} + (\alpha_{i,1} - \alpha_{i,0}) \cdot X + \cdots + (\alpha_{i,N-1} - \alpha_{i,N-2}) \cdot X^{N-1}$$

This factorization allows the computation of multiple LUTs using a single BlindRotate.

To implement MVB, it is sufficient to initialize the test polynomial *testv* with the value of v_0 during the bootstrapping phase. After performing the BlindRotate operation, the result ACC is multiplied by each v_i corresponding to the LUT of f_i to obtain ACC_i. Figure 2 illustrates the advantages of this method. This optimization reduces the number of BlindRotate operations required for an operation, thereby decreasing the overall computation time. Specifically, evaluating k LUTs using MVB requires only 1 BR instead of k separate BR.

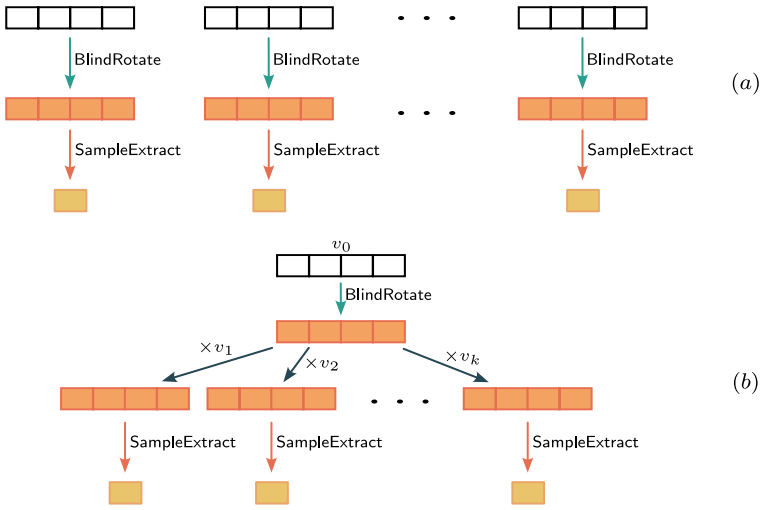


Figure 2: Illustration of the MVB optimization. (a) represents the classic method to process several bootstrapping, while (b) represents the MVB optimization. As seen here, it reduces the number of `BlindRotate` operations, which is the most expensive one of the bootstrapping. Note that once the `BlindRotate` is performed, we then work with an encrypted polynomial, hence the orange coloration. Figure inspired from [TCBS23].

3.1.3 Combining TBM and MVB.

As already noted in [GBA21], the MVB can be applied to the first level of a tree evaluation, as several BlindRotate are performed on different polynomials with the same encrypted input. For instance, regarding Figure 1, instead of requiring 5 BR and 1 KS ($\mathsf{N}_{\mathsf{BR}}= 5$ and $\mathsf{N}_{\mathsf{KS}}= 1$), the use of MVB in the first layer of the TBM results in only 2 BR (one to get the four ciphertexts associated with selector c_1, and one associated with the bootstrapping with selector c_2) and 1 KS. Note that the MVB optimization combined with the TBM does not result in fewer KS, as there are as many KS as the number of LUT being homomorphically evaluated. This way, computing the XOR of two 4-bit payload ciphertext only costs 2 BR and 1 KS. When a lookup table lut is applied on two encrypted nibbles c_0 and c_1 with such combination of MVB and TBM, we refer to the instruction $\mathsf{LUTeval}(c_0, c_1; \mathsf{lut})$ as defined in [TCB+25].

When several LUTs have to be evaluated on the same encrypted inputs, we can also use

the MVB optimization on top of the TBM. This time, we can factorize the BlindRotate of the first layer of each tree. For instance, if we want to homomorphically compute $c_0 \oplus c_1$ and $c_0 \oplus c_2$, we can factorize the bootstrappings with selector c_0 using MVB. This way, we can obtain the results of these two computations with only 3 BR (one to get the results of bootstrappings with selector c_0 in the first layers of the trees, one associated with bootstrapping with selector c_1 and one with selector c_2) and 2 KS. More generally, to compute the evaluation of k LUTs on k pairs of ciphertexts $(c_0, c_1), \cdots, (c_0, c_k)$, we can call the $\mathsf{MVLUTeval}(c_0; c_1, \cdots, c_k; \mathsf{lut}_1, \cdots, \mathsf{lut}_k)$ instruction (as defined in [TCB+25]) which costs $k+1$ BR and k KS. Without such optimization, the same evaluation of k LUTs on the same inputs would costs k calls to LUTeval, and thus $2k$ BR and k KS.

3.1.4 Evaluating an 8-bit Sbox.

As we rely on [TCBS23] to study block ciphers, we adopt the same type of ciphertexts, that is to say, we work with encryptions of nibbles (4-bit entities). We here present how to evaluate an 8-bit Sbox on 4-bit payload ciphertexts using TBM and MVB. An 8-bit message M is firstly decomposed into two 4-bit digits h and l such that $M = 16h + l$. The result of the evaluation of the Sbox `sbox` on M is thus given by $\texttt{sbox}[M] = \texttt{sbox}[16h+l] = 16h' + l'$, with $h' = \lfloor \frac{\texttt{Sbox}[M]}{16} \rfloor$ and $l' = \texttt{sbox}[M] \pmod{16}$. We thus need two tables `sbox_msn` and `sbox_lsn` such that for $h, l \in \{0, \cdots, 15\}^2$ $\texttt{sbox_msn}[16h + l] = \lfloor \frac{\texttt{Sbox}[16h+l]}{16} \rfloor$ and $\texttt{sbox_lsn}[16h+l] = \texttt{sbox}[16h+l] \pmod{16}$ to obtain the two nibbles composing the result. If we denote the encryption of M as $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$. Then, to homomorphically evaluate `sbox` on C, we have to call $\mathsf{MVLUTeval}(c_0; c_1, c_1; \texttt{sbox_msn}, \texttt{sbox_lsn})$, which costs 3 BR and 2 KS.

3.2 The Hippogryph Framework

The Hippogryph framework [BBB+25b] is a recent work that improves on the full-LUT approach, and even builds on it. We also decided to exploit this framework on top of the full-LUT approach to see if two different methods can produce the same results. The Hippogryph framework is a new approach which effectively combines the byte-oriented full-LUT approach of [TCBS23] with a generalization of the boolean-oriented p-encodings method of [BPR24]. This combination allows to capitalize on the strengths of both approaches. As a result, the authors achieve the best timings recorded to date, reducing the single-core execution time of the algorithm over TFHE from 46 seconds to 32 seconds. The p-encodings method [BPR24] encrypts data bit by bit and performs only boolean operations. It takes advantage of the fact that, in the plaintext space $\mathbb{Z}_2$, the native TFHE addition of LWE ciphertexts effectively performs XOR. This allows the linear operations of the AES (MixColumns and AddRoundKey) to be executed efficiently with minimal cost (almost for free when compared to the cost of bootstrapping) by using only the native TFHE addition. In the Hippogryph framework, the Sbox is performed with LUTs, as in [TCBS23], and the rest of the algorithm is performed with the p-encodings method. Hippogryph thus requires decompositions and recompositions of the 4-bit payload ciphertexts. These two operations that we note DC and RC have been detailed in the ISA proposed in [TCB+25] and rely on programmable bootstrapping. So, to estimate results over the Hippogryph framework, we only have to count the number of BR and KS to be executed. The decomposition of a 4-bit ciphertext c into four binary ciphertexts requires 1 BR and 0 KS(thanks to the MVB, we factorize the BlindRotate needed to obtain each bit of the decomposition) and the recomposition requires 4 BR and 0 KS (for each encryption of bit c_i, we compute with a classical bootstrapping $2^i \cdot c_i$ and then add the four new ciphertexts with the native TFHE addition). Knowing the execution timings of a BR (about 0.03 seconds) and of a KS (about 0.07 seconds) will allow us to estimate the efficiency of the framework and to draw more conclusions.

4 Running Block Ciphers over TFHE

In this section, we present each studied block cipher. First, we provide a brief overview of the specifications and structures of each cipher. Next, we explain the work done to implement them efficiently using the full-LUT approach. Building on the results from the full-LUT method, we then leverage the Hippogryph framework for a second analysis. Finally, we highlight key conclusions and findings from both studies on the block ciphers.

4.1 AES (NIST standard) [DR02]

In 2012, Gentry, Halevi, and Smart [GHS12] performed the first AES-128 homomorphic evaluation with BGV. Their runtime being of 17.5 minutes, it pushed research towards new symmetric cryptosystems designed primarily to be faster to evaluate over FHE. However, since 2023, AES homomorphic evaluations are numerous. The first work to launch what is now a friendly unofficial international competition to obtain the fastest AES execution was presented in WAHC'23 [TCBS23]. Using only TFHE's programmable bootstrapping, they obtained a sequential execution of about 4 minutes. Their approach is further called "Full-LUT" in this work. A few months later, Bon et al. proposed a new implementation running in 135 seconds and leveraging the *p-encodings* framework [BPR24]. Then, twice in a row, two papers snapped the "record" by using circuit bootstrapping in two new frameworks: Fregata [WWL+23] and Thunderbird [WLW+24]. These works brought the execution timings down to respectively 86 seconds and 46 seconds in sequential mode. In parallel, another work [SAP24] achieved a 67 secs timing using a new homomorphic LUT evaluation algorithm for binary digits relying on an adaptation of the bootstrapping to handle functions with binary range at the same computationnal cost as negacyclic functions by embedding their output in a larger domain, i.e. with only one blind rotation. Finally, in 2025, with a new framework using both the Full-LUT methodology and the *p-encodings*, Belaïd et al. managed to execute the AES in only 32 seconds in sequential mode[6], and in 1.1 seconds with some mere parallelism [BBB+25b]. In this work, we take as measurement and comparison tools both the Full-LUT approach of [TCBS23], and the Hippogryph framework of [BBB+25b].

4.2 CLEFIA

4.2.1 Specification.

CLEFIA ((ISO/IEC 29192-2 standard) [SSA+07]) is a 128-bit block cipher that supports key lengths of 128, 192, and 256 bits. It consists of two main components: a data processing part and a key scheduling part. CLEFIA uses a generalized Feistel structure with four data lines, each 32-bit wide. Additionally, it includes key whitening processes at the beginning and the end of the encryption. The number of rounds in CLEFIA varies depending on the key length: 18 rounds for a 128-bit key, 22 rounds for a 192-bit key, and 26 rounds for a 256-bit key. CLEFIA relies on two functions F_0 and F_1 that follow the same structure and use two 8-bit Sboxes in step 2. Figure 3 gives some more details about CLEFIA's specification. Fore more details, refer to [SSA+07].

4.2.2 Full-LUT Methodology.

To the best of our knowledge, this is the first attempt to evaluate CLEFIA in the homomoprhic domain. We refer the reader to Section 3.1.3 for the XOR and 8-bit Sboxes evaluations in full-LUT paradigm. What remains to be transformed into a succession

[6] This timing was obtained with a Rust implementation over `tfhe-rs`. For comparison, we work in this paper with TFHElib for which we get a slower timing of 84 seconds.

$F_0 : (RK_{(32)}, x_{(32)}) \mapsto y_{(32)}$

Step 1. $T \leftarrow RK \oplus x$
Step 2. Let $T = T_0 \mid T_1 \mid T_2 \mid T_3,\ T_i \in \{0,1\}^8$
$T_0 \leftarrow S_0(T_0)$,
$T_1 \leftarrow S_1(T_1)$,
$T_2 \leftarrow S_0(T_2)$,
$T_3 \leftarrow S_1(T_3)$
Step 3. Let $y = y_0 \mid y_1 \mid y_2 \mid y_3,\ y_i \in \{0,1\}^8$
$^t(y_0, y_1, y_2, y_3) = M_0\ ^t(T_0, T_1, T_2, T_3)$

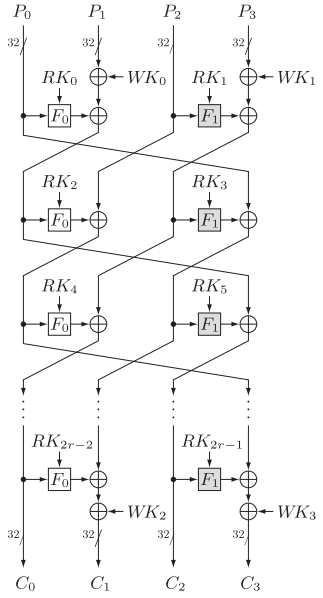


Figure 3: Figure extracted from [SSA+07].

of LUTs is the matrix multiplication in $GF(256)$. To do so, we precompute the LUTs corresponding to multiplications by `0x02`, `0x04`, `0x06`, `0x08` and `0x0a` in $GF(256)$. Indeed, all other values are not used for the multiplication. For instance, M_0 is defined as follows:

$$M_0 = \begin{bmatrix} \texttt{0x01} & \texttt{0x02} & \texttt{0x04} & \texttt{0x06} \\ \texttt{0x02} & \texttt{0x01} & \texttt{0x06} & \texttt{0x04} \\ \texttt{0x04} & \texttt{0x06} & \texttt{0x01} & \texttt{0x02} \\ \texttt{0x06} & \texttt{0x04} & \texttt{0x02} & \texttt{0x01} \end{bmatrix}$$

Each multiplication in $GF(256)$ is transformed into two tables: one that gives the most significant nibble of the multiplication (msn), and one that gives the least significant nibble of the result (lsn). For instance, from an 8-bit message $M = 16h + l$, we can obtain $M \times \texttt{0x02}$ by evaluating $16 \times \texttt{tab_mul_02_msn}[16h + l] + \texttt{tab_mul_02_lsn}[16h + l]$. In the homomorphic domain with $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$, this correspond to a call to $\mathsf{MVLUTeval}(c_0; c_1, c_1; \texttt{tab_mul_02_msn}, \texttt{tab_mul_02_lsn})$.
Then, from the four bytes defined in Figure 3 by $T_0|T_1|T_2|T_3$ we can easily obtain $y_0 = T_1 \oplus (T_2 \times \texttt{0x02}) \oplus (T_3 \times \texttt{0x04}) \oplus (T_4 \times \texttt{0x06})$. Note that each 8-bit payload ciphertext T_i will be multiplied in $GF(256)$ by the three values `0x02`, `0x04`, and `0x06`. So we can optimize the computation by calling $\mathsf{MVLUTeval}$ on each T_i. More precisely, if we note $T_i = (c_{0_i}, c_{1_i})$ where c_{0_i} and c_{1_i} are nibble encryptions, then the call corresponding to each T_i is:

$$\begin{aligned} \mathsf{MVLUTeval}(\quad & c_{0_i}; c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}; \\ & \texttt{mul_by_0x02_msn}, \texttt{mul_by_0x02_lsn}, \texttt{mul_by_0x04_msn}, \\ & \texttt{mul_by_0x04_lsn}, \texttt{mul_by_0x06_msn}, \texttt{mul_by_0x06_lsn}). \end{aligned}$$

Even better, if we create six new tables `Sbox02_msn`, `Sbox02_lsn`, `Sbox04_msn`, `Sbox04_lsn`, `Sbox06_msn` and `Sbox06_lsn`, corresponding to the Sbox respectively multiplied by `0x02`, `0x04` and `0x06`, we can factorize calls to $\mathsf{MVLUTeval}$ one step further. Indeed, as the matrix multiplication directly follows the SBox evaluation, we can precompute the multiplication following the Sbox. Then, for each T_i we perform only one call to $\mathsf{MVLUTeval}$ that will give us all the values needed to compute step 2 and step 3 of F_0. For one T_i this one call

would give $\texttt{sbox}[T_i]$, $\texttt{sbox}[T_i] \times \texttt{0x02}$, , $\texttt{sbox}[T_i] \times \texttt{0x04}$ and , $\texttt{sbox}[T_i] \times \texttt{0x06}$. Such a call can be defined the following way:

$$\begin{aligned}\mathsf{MVLUTeval}(\quad & c_{0_i}; c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}; \\ & \texttt{Sbox_msn}, \texttt{Sbox_lsn}, \texttt{Sbox02_msn}, \texttt{Sbox02_lsn}, \\ & \texttt{Sbox04_msn}, \texttt{Sbox04_lsn}, \texttt{Sbox06_msn}, \texttt{Sbox06_lsn}).\end{aligned}$$

Thus, to obtain all the terms needed to obtain the y at the end of an F_0 evaluation, it requires $4 \times (9\ \mathsf{BR}, 8\ \mathsf{KS}) + 4 \times 3 \times 2 \times (2\ \mathsf{BR}, 1\ \mathsf{KS}) = 84\ \mathsf{BR}$ and $56\ \mathsf{KS}$ instead of $4 \times (3\ \mathsf{BR}, 2\ \mathsf{KS}) + 4 \times (7\ \mathsf{BR}, 6\ \mathsf{KS}) + 4 \times 3 \times 2 \times (2\mathsf{BR}, 1\ \mathsf{KS}) = 88\ \mathsf{BR}$ and $56\ \mathsf{KS}$.

With such an optimization, one complete evaluation of F_0 costs 100 BR and 64 KS (if we add the counts for XORs on 32-bit words in step 1 of F_0). Thus a whole evaluation, containing 36 calls to F_i and 40 XORs would cost 4240 BR and 2624 KS, which takes about 311 seconds.

4.2.3 Hippogryph Approach.

We assume there exist, as for the AES, a boolean circuit corresponding to the matrix multiplication, which allows to compute this state nearly for free with the Hippogryph framework. Indeed, even though CLEFIA and AES do not use the same irreducible polynomial, the evaluation method remains the same[7]. With such framework, the only bootstrappings to compute are the ones for the Sboxes, and the ones to decompose (DC) and recompose (RC) the messages.
We perform RC at the beginning of an F_i on 4 bytes, thus costing $4 \times 2 \times (4\ \mathsf{BR}, 0\ \mathsf{KS})$. After step 2, we perform DC again on 4 bytes, costing $4 \times 2 \times (1\ \mathsf{BR}, 0\ \mathsf{KS})$. And of course, we perform the Sbox at step 2 on each byte, which costs $4 \times (3\ \mathsf{BR}, 2\ \mathsf{KS})$. Thus, a full F_i execution costs a total of 52 BR and 8 KS, and 36 F_i executions cost 1872 BR and 288 KS. Estimated timings are given in Table 1.

4.2.4 Concluding Remarks.

Always look everywhere for small optimization. For instance, here our trick saves us 4 BR by F_i. It can seem to be nothing, but on a complete execution, it saves us 144 BR, that is to say about 4.5 seconds. Finally, let us note that with the Hippogryph approach, CLEFIA seems to be more efficient than the AES.

4.3 PRESENT

4.3.1 Specification.

PRESENT (ISO/IEC 29192-2 standard) [BKL+07] is an example of a Substitution-Permutation network consisting of 31 rounds working on a 64-bit state matrix (meaning that we have to compute two executions of PRESENT to obtain 128 bits). Each round includes an XOR operation that incorporates a round key (K_i) for ($1 \leq i \leq 32$), with (K_{32}) being used for post-whitening. Each round also features a linear bitwise permutation and a non-linear substitution layer. The non-linear layer utilizes a single 4-bit S-box, which is applied on the 16 nibbles of the state during each round. Figure 4 presents the bit permutation of the cipher, also called *pLayer*. The execution of PRESENT can be resumed with the following algorithm:

[7] We do not assume that the circuit for matrix multiplication in CLEFIA is the same as that for AES, but rather that one exists. In this case, this circuit would be evaluated for free with a 2-encoding, as in the case of AES with Hippogryph.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$P(i)$	0	16	32	48	1	17	33	49	2	18	34	50	3	19	35	51
i	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
$P(i)$	4	20	36	52	5	21	37	53	6	22	38	54	7	23	39	55
i	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
$P(i)$	8	24	40	56	9	25	41	57	10	26	42	58	11	27	43	59
i	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
$P(i)$	12	28	44	60	13	29	45	61	14	30	46	62	15	31	47	63

Figure 4: The bit permutation used in PRESENT: bit i of state is moved to bit position $P(i)$. This figure is extracted from [BKL+07].

generateRoundKeys()
for $i = 1$ **to** 31 **do**
    addRoundKey(STATE,K_i)
    sBoxLayer(STATE)
    pLayer(STATE)
addRoundKey(STATE,K_{32})

4.3.2 Full-LUT Methodology.

To the best of our knowledge, this is the first attempt to evaluate PRESENT in the homomorphic domain. We have already seen how to compute XORs with LUTs (see Section 3.1.3). Note that the Sbox of PRESENT is a 4-bit Sbox, so there is no need for the TBM to evaluate it with our framework. Indeed, as we work with 4-bit payload ciphertext, a classical bootstrapping is sufficient to evaluate the Sbox on encryptions of nibbles. This is great, as the evaluation of the Sbox on an 8-bit payload ciphertext then costs only 2 BR, instead of 3 BR and 2 KS for an 8-bit Sbox. That is to say, it takes about 0.06 seconds to evaluate instead of about 0.23 seconds.
The really difficult part to process is the so-called *pLayer* of PRESENT. This pLayer is a bit permutation that is hard to compute with 4-bit chunks. In fact, the most efficient way to homomorphically compute it with 4-bit payload ciphertexts is to decompose them into binary ciphertexts, perform the permutation on these binary ciphertexts and then recompose the nibbles. Thus, the permutation itself can be seen as "free" (we only move the binary ciphertexts), but we have to compute several decompositions and recompositions. A decomposition (DC) takes 1 BR and 0 KS per nibble, and a recomposition (RC) takes 4 BR and 0 KS. These are rather cheap operations as they do not rely on TBM, but only on simple bootstrapping.
So one round thus costs 16 homomorphic XORs, 16 4-bit Sbox evaluations and 16 DC/RC to compute the pLayer. That is to say, one round costs $16 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (0\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (4\ \mathsf{BR}, 0\ \mathsf{KS}) = 128\ \mathsf{BR}$ and 16 KS. As there are 31 rounds, a complete execution costs 3968 BR and 288 KS and takes about 154 seconds. But as it only gives a 64-bit results, we have to compute the full execution twice to obtain 128 bits.

4.3.3 Hippogryph Approach.

With the Hippogryph framework, only the Sbox is computed with TFHE's programmable bootstrapping. So we have to recompose 4-bit payload ciphertexts to evaluate the Sbox and then to decompose them to perform the pLayer and the XOR with the round keys. Note that even in the full-LUT paradigm, we have to perform these decompositions and recompositions to efficiently perform the pLayer.
Thus one round costs $16 \times (4\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) = 96\ \mathsf{BR}$

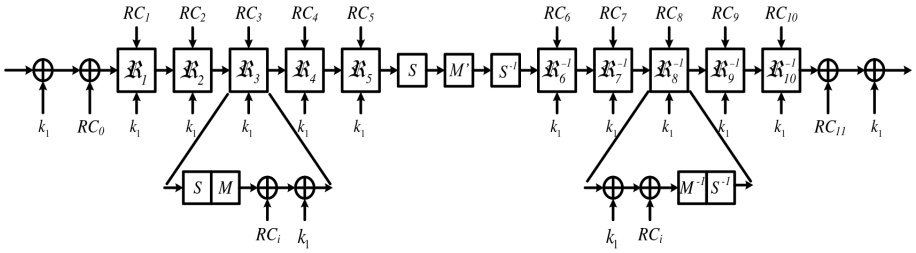


Figure 5: PRINCE$_{core}$, reproduced from [BCG+12].

and 0 KS. Thus a complete evaluation requires 2976 BR and 0 KS. Estimated timings are given in Table 1.

4.3.4 Concluding Remarks.

What we learn from PRESENT is that to be efficient in the homomorphic domain, a cipher must have a good trade-off between the number of operations per rounds and the number of rounds. For instance, here a round is really light, even in the homomorphic domain (it takes less than 5 seconds in full-LUT version, and less than 3 seconds with Hippogryph), but we have to repeat it 31 times. If it was only 10 such rounds –as for the AES–, it would cost 1280 BR and 160 KS (about 50 seconds) in full-LUT mode, or 960 BR and 0 KS (about 29 seconds) with the Hippogryph framework.

4.4 PRINCE

4.4.1 Specification.

PRINCE is a 64-bit block cipher that uses a 128-bit key. The key is divided into two parts of 64 bits each, represented as $k = k_0||k_1$ (with $||$ the symbol for concatenation), and is extended to 192 bits through the mapping $(k_0||k_1) \rightarrow (k_0||k'_0||k_1) := (k_0||(k_0 \gg 1) \oplus (k_0 \ll 63)||k_1)$. The first two subkeys, k_0 and k'_0, serve as whitening keys, while the key k_1 is the 64-bit key for a 12-round block cipher known as PRINCE$_{core}$. Each round of PRINCE$_{core}$ includes a key addition, an S-box layer, a linear layer, and the addition of a round constant. Figure 5 presents the PRINCE$_{core}$ features. During the key addition phase, the 64-bit state is XORed with the 64-bit key k_1. The S-layer utilizes a single 4-bit S-box. The cipher incorporates M and M' layers, where the 64-bit state is multiplied by a 64×64 bits matrix M (or M'). Lastly, in the Round Constant addition step, a 64-bit round constant RC is XORed with the state.

4.4.2 Full-LUT Methodology.

PRINCE has already been studied for homomorphic purposes in the past. For instance, Doröz et al. [DSES14] proposed an implementation relying on a generalization of NTRU that took 57 minutes to execute. More recently, Bendoukha et al. [BSS+23] also studied the PRINCE block cipher. For an optimized full-LUT evaluation, the original PRINCE algorithm must be carefully analyzed. The first intuition when looking at the algorithm is that the two XORs (one of the state with a round constant, and one with a round key) can be factorized into one single XOR. To do so, the client can compute the XOR of the round constants with the corresponding round keys, obtaining a 64-bit entity that we denote $RCk_i = k_1 \oplus RC_i$. As these computations are done in the cleartext space, this can be done very efficiently. Then instead of sending FHE encryptions of the round keys to

the server, the client will send FHE encryptions of the result of the XOR of the round keys with the round constants, which represents the same amount of data. Lastly, we remind that before the execution of PRINCE$_{core}$, the 64-bit state must be XORed with the 64-bit key k_0, and at the end of PRINCE$_{core}$, the resulting 64-bit state is XORed with the 64-bit key k'_0. Note that these additional XORs do not impact our homomorphic execution of the framework. Indeed, as the first XOR with k_0 directly precedes the first XOR of PRINCE$_{core}$, the client can precompute $RCk'_0 = k_0 \oplus k_1 \oplus RC_0$ to replace RCk_0 and send it to the server along with the other RCk_i. Similarly, as the XOR with the 64-bit key k'_0 is the last operations and directly follows the XOR of the state with RCk_{11}, the client can precompute $RCk'_{11} = k'_0 \oplus k_1 \oplus RC_{11}$ and send it to the server instead of RCk_{11}. With this small optimization, the number of homomorphic XOR remains the same, and we even managed to decrease the number of FHE encryptions to send to the server.

Then, what remains to do is to transform the linear layer, consisting of a permutation similar to the AES ShiftRows and a matrix multiplication into a succession of LUTs. As the permutation permutes nibble, it is free in our design. So the one thing that really needs to be translated into LUTs is the matrix multiplication. We only consider here an approach which consists in replacing each of the 16 4×4 submatrices of $\hat{M}^{(0)}$ and $\hat{M}^{(1)}$ into a single LUT evaluation (i.e. accounting for 16 LUT evaluations in total, see Fig. 6). We experimentally observed that this was the most efficient choice among other packing options, e.g. either over rows or columns. This Boolean matrix multiplication is the challenging part that makes PRINCE not as "FHE-friendly" as one could have expected.

We note $c = (c_0c_1c_2c_3c_4c_5c_6c_7c_8c_9c_{10}c_{11}c_{12}c_{13}c_{14}c_{15})$ the encryption of the 64-bit state where each c_i is the encryption of a nibble. We want to compute the multiplication of c with M', with M' a 64×64 diagonal matrix which diagonal blocks are $(\hat{M}^{(0)}, \hat{M}^{(1)}, \hat{M}^{(1)}, \hat{M}^{(0)})$. To do so, we create from the 16×16 bits matrices $\hat{M}^{(0)}$ and $\hat{M}^{(1)}$ the 16×16 bits matrices $\tilde{M}^{(0)}$ and $\tilde{M}^{(1)}$ by computing $\tilde{c}_{i,j} = c_i \wedge (15 - 2^j)$ for $i \in \{0, 1, \cdots, 15\}$ and $j \in \{0, 1, 2, 3\}$. Figure 6 shows $\hat{M}^{(0)}$ and $\tilde{M}^{(0)}$.
For instance, with such matrices, the evaluation of the multiplication on c_0 is calculated as follows:

$$\bar{c}_0 = \tilde{c}_{0,0} \oplus \tilde{c}_{1,1} \oplus \tilde{c}_{2,2} \oplus \tilde{c}_{3,3}.$$

That is to say, to obtain the encryption of the evaluation of the matrix multiplication on a nibble, we have to compute the corresponding $\tilde{c}_{i,j}$ and perform three homomorphic XORs. Each $\tilde{c}_{i,j}$ is computed once, with a simple bootstrapping. Note that to compute all $\tilde{c}_{i,i}$, $\tilde{c}_{i,(i+1) \pmod 4}$, $\tilde{c}_{i,(i+2) \pmod 4}$, $\tilde{c}_{i,(i+3) \pmod 4}$, we have to bootstrap with selector c_i. This means we can factorize these bootstrappings with the MVB an obtain four $\tilde{c}_{i,j}$ at the cost of only one BR. This way, the complete multiplication requires one BR per nibble to compute the new matrices (that is to say 16 BR in total) and then three XORs per nibble to actually perform the multiplication (i.e. $3 \times (2$ BR, 1 KS$) = 6$ BR and 3 KS per nibble). So, a complete multiplication requires 112 BR and 48 KS.
A full round composed of the Sbox, the linear layer and the XOR with the new round keys then costs 160 BR and 64 KS. The whole decryption process of PRINCE comprises ten such rounds, one XOR at the beginning of the process and one at the end of the process, and two Sbox evaluation separated by a matrix multiplication (as shown in Figure 5), costing 1808 BR and 720 KS.

4.4.3 Hippogryph Approach.

With the Hippogryph framework, we only need the 4-bit payload ciphertexts for the evaluation of the Sbox. We perform multiplications and XORs with binary ciphertexts. As we perform twelve evaluation of the Sbox; we have to perform twelve RC and twelve DC,

$$\hat{M}^{(0)} = \begin{bmatrix}
0\,0\,0\,0 & 1\,0\,0\,0 & 1\,0\,0\,0 & 1\,0\,0\,0 \\
0\,1\,0\,0 & 0\,0\,0\,0 & 0\,1\,0\,0 & 0\,1\,0\,0 \\
0\,0\,1\,0 & 0\,0\,1\,0 & 0\,0\,0\,0 & 0\,0\,1\,0 \\
0\,0\,0\,1 & 0\,0\,0\,1 & 0\,0\,0\,1 & 0\,0\,0\,0 \\
\\
1\,0\,0\,0 & 1\,0\,0\,0 & 1\,0\,0\,0 & 0\,0\,0\,0 \\
0\,0\,0\,0 & 0\,1\,0\,0 & 0\,1\,0\,0 & 0\,1\,0\,0 \\
0\,0\,1\,0 & 0\,0\,0\,0 & 0\,0\,1\,0 & 0\,0\,1\,0 \\
0\,0\,0\,1 & 0\,0\,0\,1 & 0\,0\,0\,0 & 0\,0\,0\,1 \\
\\
1\,0\,0\,0 & 1\,0\,0\,0 & 0\,0\,0\,0 & 1\,0\,0\,0 \\
0\,1\,0\,0 & 0\,1\,0\,0 & 0\,1\,0\,0 & 0\,0\,0\,0 \\
0\,0\,0\,0 & 0\,0\,1\,0 & 0\,0\,1\,0 & 0\,0\,1\,0 \\
0\,0\,0\,1 & 0\,0\,0\,0 & 0\,0\,0\,1 & 0\,0\,0\,1 \\
\\
1\,0\,0\,0 & 0\,0\,0\,0 & 1\,0\,0\,0 & 1\,0\,0\,0 \\
0\,1\,0\,0 & 0\,1\,0\,0 & 0\,0\,0\,0 & 0\,1\,0\,0 \\
0\,0\,1\,0 & 0\,0\,1\,0 & 0\,0\,1\,0 & 0\,0\,0\,0 \\
0\,0\,0\,0 & 0\,0\,0\,1 & 0\,0\,0\,1 & 0\,0\,0\,1
\end{bmatrix} \rightarrow \tilde{M}^{(0)} = \begin{bmatrix}
\tilde{c}_{0,0} & \tilde{c}_{0,1} & \tilde{c}_{0,2} & \tilde{c}_{0,3} \\
\tilde{c}_{1,1} & \tilde{c}_{1,2} & \tilde{c}_{1,3} & \tilde{c}_{1,0} \\
\tilde{c}_{2,2} & \tilde{c}_{2,3} & \tilde{c}_{2,0} & \tilde{c}_{2,1} \\
\tilde{c}_{3,3} & \tilde{c}_{3,0} & \tilde{c}_{3,1} & \tilde{c}_{3,2}
\end{bmatrix}$$

Figure 6: $\hat{M}^{(0)}$ is part of the matrix multiplication of PRINCE. To implement it more efficiently, we transform it into the matrix $\tilde{M}^{(0)}$.

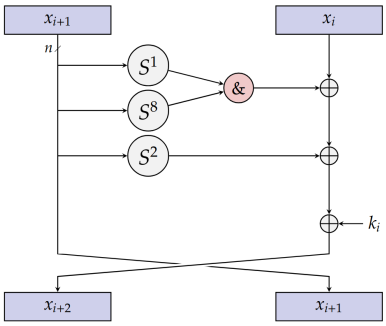


Figure 7: Feistel stepping of the SIMON round function extracted from [BTCS+15].

as well as 16 BR per Sbox evaluation. That is to say, we have to compute $12 \times 16 \times (5$ BR, 0 KS$) + 12 \times (16$ BR, 0 KS$) = 1152$ BR and 0 KS. Execution timings are given in Table 1.

4.4.4 Concluding Remarks.

Once again, there was an easy way to optimize the algorithm by factorizing the XORs of the round keys with those of the round constants, as well as the pre and post PRINCE$_{core}$ XORs. Also, there exist several ways to transform an operation into a succession of LUTs, so we have to try several times before we find the most optimized one. Finally, as for PRESENT, the 4-bit Sbox is a considerable advantage as it does not require the use of the TBM.

4.5 SIMON

4.5.1 Specification.

The SIMON (ISO/IEC 29167-21 standard) [BTCS+15] block cipher operates on pairs of n-bit words, which results in a total block size of $2n$ bits. The value of n can be 16, 24, 32, 48, or 64 bits. For our comparison with AES, we will use 64-bit words.

SIMON utilizes a Feistel structure that only involves left circular shifts (denoted S^i for a shift of an index i), XOR operations, and AND operations on n-bit words. For the 64-bit version, the encryption process requires the execution of 68 rounds, as summarized in Figure 7. As SIMON does not require any Sbox evaluation, the full-LUT approach will not perform well. Several attempts to evaluate SIMON in the homomorphic domain can be cited. For instance, a few years ago, Lepoint and Naehrig proposed the first implementation in leveled mode, which achieved a 12 minutes latency [LN14]. More recently, Bon et al.

[BPR24] studied SIMON and other primitives as use cases for the *p-encodings* approach. They report a 10 seconds latency.

4.5.2 Full-LUT Methodology.

With the full-LUT approach, homomorphic AND evaluations are conducted in a manner similar to homomorphic XOR evaluations (see Section 3.1.3), but utilizing different tables. However, implementing circular shifts poses a greater challenge.

To clarify, for circular shifts, also called rotations, bits that are "shifted out" are reinserted either at the beginning or the end of the word, depending on the direction of the shift. When it comes to executing cleartext-ciphertext rotations (that is to say rotating an encrypted value using a cleartext index), we can optimize the operation on a case-by-case basis. For example, a shift by 4 or any multiple of 4 is nearly free to compute since it involves shifting the entire 4-bit payload ciphertext. In contrast, shifting by an index that is not a multiple of 4 requires additional effort. In the work presented in [TCB+25], the authors propose a set of operations that leverage TFHE's programmable bootstrapping, including an implementation for circular shifts, which we adopt here. The proposed solution is designed for the encryption of bytes divided into two nibbles. To execute a circular shift on an encrypted byte, the authors present an algorithm that requires only 4 BR (two per nibble). One bootstrapping corresponds to the portion that "remains" within the nibble, and the other is for the part that is "shifted out" of the nibble. A SIMON round is composed of three left circular shifts S^1, S^2 and S^8. As 8 is a multiple of 4, we can perform it without any bootstrapping. But to shift with indexes 1 and 2, we have to use the algorithm discuss above. To shift the encryption of one byte, it takes 4 BR. Here, we work with words of 64 bits, that is to say 8 bytes. So to perform one circular shift on 8 bytes, we need to compute $8 \times (4\ \mathsf{BR}, 0\ \mathsf{KS}) = 32\ \mathsf{BR}$ and $0\ \mathsf{KS}$. So to perform S^1 and S^2 (recall that S^8 is free in our full-LUT framework), we have to perform 64 BR. In Section 3.1.3, we determined that performing a bitwise operand on a encrypted nibble requires 2 BR and 1 KS. So performing the three XORs and the AND on SIMON's 64-bit words costs $4 \times 16 \times (2\ \mathsf{BR}, 1\ \mathsf{KS}) = 128\ \mathsf{BR}$ and 64 KS.
A full round thus costs 192 BR and 64 KS. As there are 68 such rounds, a complete execution costs 13056 BR and 4352 KS. Execution timings are given in Table 1.

4.5.3 Hippogryph Approach.

Note that as SIMON does not rely on Sbox evaluation, there is no need to use 4-bit payload ciphertexts, thus no decomposition or recomposition are needed. So the Hippogryph framework is not really needed, and only some bootstrappings are performed to refresh the ciphertexts (as 68 rounds imply a circuit depth too large to handle with reasonable parameters). This of course induce a super fast evaluation of SIMON with binary ciphertexts for instance with the p-encoding method [BPR24]. For these reasons, we do not simulate the Hippogryph framework for SIMON.

4.5.4 Concluding Remarks.

If the block cipher used for transciphering does not rely on Sbox evaluation, then it is more advantageous to use binary-encoding ciphertexts and Boolean circuits approaches.

4.6 SKINNY

4.6.1 Specification.

The lightweight block ciphers of the SKINNY family (ISO/IEC 18033-7 standard) [BJK+16] have 64-bit and 128-bit block versions. In both $n = 64$ and $n = 128$ versions, the internal

state is viewed as a 4×4 square array of cells, where each cell is a nibble (in the 64-bit case) or a byte (in the 128-bit case). In the 64-bit variant, 32 rounds are applied to the state, when 40 are applied in the 128-bit variant. Another difference between the two variants is the Sbox. In the 128-bit variant, the authors propose to use an 8-bit Sbox, and for the 64-bit variant they propose a 4-bit Sbox. One round is composed of five operations: an Sbox, two XORs (one with a round constant and one with a round tweakey), a shift of the rows of the state and a matrix multiplication.

4.6.2 Full-LUT Methodology.

To the best of our knowledge, this is the first attempt to evaluate SKINNY in the homomorphic domain. In both variants, we optimize the algorithm by factorizing the two different XORs being applied to the state. As for PRINCE, the client pre-computes the XOR of each round constant with the corresponding keys and send these modified keys to the server. This saves us several bootstrappings.
As with other block ciphers, the most difficult part is to convert the matrix multiplication into a succession of LUT evaluations. But in the SKINNY case, the matrix is a binary matrix, meaning that we only have to correctly XOR the corresponding values to perform the matrix multiplication. More precisely, to obtain four nibbles (resp. four bytes), we have to compute four XORs. This means that the whole multiplication only cost 16 XORs. It is the cheapest matrix multiplication so far.
In the 64-bit case (resp. 128-bit case), the multiplication thus costs $16 \times (2\ \mathsf{BR}, 1\ \mathsf{KS}) = 32\ \mathsf{BR}$ and $16\ \mathsf{KS}$ (resp. $16 \times 2 \times (2\ \mathsf{BR}, 1\ \mathsf{KS}) = 64\ \mathsf{BR}$ and $32\ \mathsf{KS}$). For the Sbox step, we have to perform $16\ \mathsf{BR}$ in the 64-bit version, against $48\ \mathsf{BR}$ and $32\ \mathsf{KS}$ in the 128-bit version. Finally, homomorphically compute the XOR layer requires $16 \times (2\ \mathsf{BR}, 1\ \mathsf{KS}) = 32\ \mathsf{BR}$ and $16\ \mathsf{KS}$ for the 64-bit variant and $16 \times 2 \times (2\ \mathsf{BR}, 1\ \mathsf{KS}) = 64\ \mathsf{BR}$ and $32\ \mathsf{KS}$ for the 128-bit variant.
Thus, a full round of SKINNY-64 costs $80\ \mathsf{BR}$ and $32\ \mathsf{KS}$, and a full round of SKINNY-128 costs $176\ \mathsf{BR}$ and $96\ \mathsf{KS}$. Even if we have to perform two full execution of SKINNY-64 to obtain 128 encrypted bits, it is less costly to run in the homomorphic domain than one complete execution of SKINNY-128. Execution timings are given in Table 1.

4.6.3 Hippogryph Approach.

Note that as with PRINCE, the optimization of the factorization of XORs also benefits the Hippogryph framework. Indeed, it allows a smaller depth of Boolean circuit to be performed, and thus smaller parameters.
Still, the Hippogryph framework requires 4-bit payload ciphertexts to compute the SBox with PBS. So the only bootstrappings to perform are those corresponding to the decomposition of ciphertexts (DC), recomposition of ciphertexts (RC) and Sbox evaluation. There is one Sbox evaluation per round regardless of the variant being executed. So, for one round in the 128-bit case, we have to compute $16 \times 2 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times 2 \times (4\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (3\ \mathsf{BR}, 2\ \mathsf{KS}) = 208\ \mathsf{BR}$ and $32\ \mathsf{KS}$. One round of the 64-bit version costs $16 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (4\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) = 96\ \mathsf{BR}$ and $0\ \mathsf{KS}$. These are quite light rounds, but to perform the complete execution, we have to repeat them 40 times for the 128-bit version, and 2×32 times to obtain 128 bits with the 64-bit version.

4.6.4 Concluding Remarks.

An interesting fact to note is that, with both variants, the Hippogryph framework requires more BR than in the full-LUT paradigm, but remains more efficient (between $0.6\times$ and $0.7\times$ faster than the full-LUT method). It is due to the small number of KS required. While one BR takes about 0.03 seconds, one KS takes about 0.07 seconds which is almost

twice as the execution time of one BR. So the greater number of BR in the Hippogryph Approach is cushioned by the absence of KS, which ultimately makes the Hippogryph approach more interesting. We also note that the matrix multiplication costs is very small compared to other block ciphers. And finally, it is very interesting to see that to perform twice the 64-bit variant is almost 2× faster than one execution of the 64-bit variant. As the operations are the same (XORs, Sbox, Shifts,...), the explanation comes from the fact that they are applied on nibbles and not on bytes, which makes them way much faster (very fewer BR, and almost no KS!).

5 Results

Our results with the full-LUT approach, presented in Table 1, are obtained with a 12th Gen Intel(R) Core(TM) i7-12700H CPU laptop with 64 Gib total system memory with an Ubuntu 22.04.2 LTS server, using a single core. The estimated timings for the Hippogryph framework are given in the same table, for the same set of parameters. For reproducibility of our results, we have provided the source code on Github at https://github.com/daphnetrm/Running-Standard-Block-Ciphers-with-TFHE.With our generic full-LUT method, we show that AES is not the fastest symmetric block cipher. Indeed, PRINCE is faster with both the Full-LUT approach and the Hippogryph framework. Note that the Hippogryph framework, CLEFIA is also faster than the AES.

5.1 Several Categories of Instructions

Using these executions timings and estimated timings, we can draw several additional conclusions. For instance, based on our execution times, we have identified which operations are relatively fast in the homomorphic domain and which are more costly. FHE-friendly features include working on sub-block messages of 4 or 8 bits rather than directly binary messages. In particular, when shifting the rows of the state matrix, we prefer shifts with an index multiple of 4 (and therefore free in our FHE design). A trade-off must also be found between the number of bootstrapping operations per round and the total number of rounds. Finally, a 4-bit Sbox is more than two times faster than an 8-bit Sbox. Note that this final FHE-friendly feature as been recently highlighted in [BBB+25a], where the authors propose a new stream cipher especially designed to be efficient for transciphering with TFHE.

On the FHE-unfriendly side, we have 8-bit Sboxes, lots of ciphertext-to-ciphertext operations (more costly than univariate operations, for instance, the multiplication by 2 in $GF(256)$ in AES' MixColumns), index shifts that are not multiples of 4, a very large number of rounds, etc. We also emphasize one advantage of block ciphers (compared to stream ciphers): the possibility of parallelizing computations. Indeed, for instance with AES calculations can be performed on each byte separately (except for MixColumns), as with PRINCE, PRESENT, and SKINNY. As for Feistel-type block ciphers, such as SIMON and CLEFIA, we can parallelize by the number of words. We have not used this advantage here, as we are mainly interested in comparing the sequential execution times of different schemes.

5.2 A Fantasy FHE-friendly Block Cipher

Finally, this categorization of block ciphers' operations allows us to highlight the ideal building blocks for a hypothetical Unicorn Cipher that would outperform other ciphers in the homomorphic domain:

Table 1: Comparison of different block ciphers running over TFHE. Timings are in seconds. Results are provided for the Full-LUT and Hippogryph methods. For each method we have provided the number of Blind Rotations ($\mathsf{N_{BR}}$) and Key Switches ($\mathsf{N_{KS}}$) as these two operations dominate by far the overall computational cost.

Cryptosystems	Nb bits in state	nb executions to obtain 128 bits	Full-LUT			Hippogryph		
			$\mathsf{N_{BR}}$	$\mathsf{N_{KS}}$	timings to obtain 128 bits	$\mathsf{N_{BR}}$	$\mathsf{N_{KS}}$	timings to obtain 128 bits
AES	128	1	3776	2112	260	2080	320	84
PRINCE	64	2	1808	1220	210	1152	0	70
PRESENT	64	2	4496	992	408	2976	0	179
SKINNY-128	128	1	7040	3840	480	8192	1280	335
SKINNY-64	64	2	2560	1024	298	3008	0	180
CLEFIA	128	1	5120	2848	353	1872	288	77
SIMON	128	1	18496	4080	840	-	-	-
Unicorn	128	1	1600	640	93	1920	0	58

- Subsitution layer: evaluation of a 4-bit Sbox on the state (for example PRINCE's Sbox)
- Permutation layer: SKINNY's MixColumns that only requires XORs (no multiplications).
- Linear layer: AES's ShiftRows that shifts from indexes that are multiples of 4.
- Round Key addition: a simple XOR of the state with the corresponding round key. Note that we can also add a Round Constant Addition operation for free if we factorize it with the Round Key Addition.

Without professing any symmetric encryption security, a 128-bit block size Unicorn Cipher which round tours would be composed of these four operations would only cost 160 BR and 64 KS per round with the full-LUT approach, and 192 BR and 0 KS with the Hippogryph framework. Last line of Table 1 resumes the number of BR and KS needed for such a Unicorn cipher that would require ten rounds to execute. We also propose the estimated timings for the two approaches.

Interestingly, the recent Transistor FHE-friendly stream-cipher design [BBB+25a] independently follows guidelines which are consistent with the above ones.

6 Conclusion

This work shows that, with two different approaches, there exist standardized block ciphers that can be more efficiently used for transciphering than AES (namely PRINCE with the full-LUT methodology and PRINCE and CLEFIA with the Hippogryph framework). But above all, this work points to two avenues of research that could contribute to efficient transciphering (and more) over TFHE:

- Focus on the operations categorized here as "FHE-unfriendly" and create more efficient FHE operators for them.
- Investigate whether secure block ciphers can be built using only the "FHE-friendly" building-blocks that we identified.

Finally, let us emphasize that the method to be used for transciphering does not necessarily have to be the most efficient. Indeed, it should also depend on the type of homomorphic

calculations that are being performed after the transciphering. For instance, functional bootstrapping methods generally produce ciphertexts with integer-based encodings, while circuit bootstrapping uses bit encodings. Besides minor latency differences, the choice between integer and bit outputs has a significant impact on further homomorphic evaluation, particularly regarding compatibility with arithmetic versus Boolean circuits. This trade-off indicates that the best approach may depend on the specific use case and the types of subsequent operations involved.

A Parameters

We present here the parameter set we used for our implementations with TFHElib.

Table 2: Parameters set proposed in [TCB+25]. B_g and l denote the basis and levels associated with the gadget decomposition, B_{KS} and t denote the decomposition basis and the precision of the decomposition of the PublicFunctionalKeyswitch, r denotes the plaintext modulus, and ϵ is the error probability of one MVB tree-based evaluation.

n	q	N	l	B_g	B_{KS}	t	r	ϵ	TRLWE std	TLWE std
1024	2^{32}	2048	3	256	1024	2	32	2^{-23}	9.6×10^{-11}	6.5×10^{-8}

References

[AMT22] T. Ashur, M. Mahzoun, and D. Toprakhisar. Chaghri - a fhe-friendly block cipher. In *ACM CCS*, page 139–150, 2022. doi:10.1145/3548606.3559364.

[ARS+15] M. Albrecht, C. Rechberger, T. Schneider, T. Tiessen, and M. Zohner. Ciphers for MPC and FHE. In *EUROCRYPT*, pages 430–454, 2015. doi:0.1007/978-3-662-46800-5_17.

[BBB+25a] J. Baudrin, S. Belaïd, N. Bon, C. Boura, A. Canteaut, G. Leurent, P. Paillier, L. Perrin, M. Rivain, Y. Rotella, and S. Tap. Transistor: a TFHE-friendly stream cipher. In *CRYPTO*, page 530–565, 2025. doi:10.1007/978-3-032-01901-1_17.

[BBB+25b] Sonia Belaïd, Nicolas Bon, Aymen Boudguiga, Renaud Sirdey, Daphné Trama, and Nicolas Ye. Further improvements in AES execution over TFHE. *IACR Communications in Cryptology*, 2(1), 2025. doi:10.62056/ahmp-4tw9.

[BBS21] A.-A. Bendoukha, A. Boudguiga, and R. Sirdey. Revisiting stream-cipher-based homomorphic transciphering in the tfhe era. In *Foundations and Practice of Security: 14th International Symposium, FPS*, page 19–33, 2021. doi:10.1007/978-3-031-08147-7_2.

[BCBS23] A.-A. Bendoukha, P.-E. Clet, A. Boudguiga, and R. Sirdey. Optimized stream-cipher-based transciphering by means of functional-bootstrapping. In *Data and Applications Security and Privacy XXXVII: 37th Annual IFIP WG 11.3 Conference,*, page 91–109, 2023. doi:10.1007/978-3-031-37586-6_6.

[BCG+12] J. Borghoff, A. Canteaut, T. Güneysu, E. Bilge Kavun, M. Knezevic, L. Ramkilde Knudsen, G. Leander, V. Nikov, C. Paar, C. Rechberger, P. Rombouts, S. S. Thomsen, and T. Yalçin. PRINCE - a low-latency block cipher for pervasive computing applications. In *ASIACRYPT*, page 208–225, 2012. doi:0.1007/978-3-642-34961-4_14.

[BJK+16] C. Beierle, J. Jean, S. Kölbl, G. Leander, A. Moradi, Thomas Peyrin, Y. Sasaki, P. Sasdrich, and S. M. Sim. The SKINNY family of block ciphers and its low-latency variant MANTIS. In *CRYPTO*, pages 123–153, 2016. doi:10.1007/978-3-662-53008-5_5.

[BKL+07] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Vikkelsoe. Present: An ultra-lightweight block cipher. In *CHES*, pages 450–466, 2007. doi:10.1007/978-3-540-74735-2_31.

[BOS23] T. Balenbois, J.-B. Orfila, and N. Smart. Trivial transciphering with trivium and tfhe. In *WAHC*, 2023. doi:10.1145/3605759.3625255.

[BPR24] N. Bon, D. Pointcheval, and M. Rivain. Optimized homomorphic evaluation of boolean functions. *IACR TCHES*, 2024. doi:10.46586/tches.v2024.i3.302-341.

[BSS+23] A. A. Bendoukha, O. Stan, R. Sirdey, N. Quero, and L. Freitas. Practical homomorphic evaluation of block-cipher-based hash functions with applications. In *FPS*, pages 88–103, 2023. doi:10.1007/978-3-031-30122-3_6.

[BTCS+15] R. Beaulieu, S. Treatman-Clark, D. Shors, B. Weeks, Jason S., and L. Wingers. The SIMON and SPECK lightweight block ciphers. In *DAC*, 2015. doi:10.1145/2744769.2747946.

[CBSZ23] P.-E. Clet, A. Boudguiga, R. Sirdey, and M. Zuber. Combo: A novel functional bootstrapping method for efficient evaluation of nonlinear functions in the encrypted domain. In *AFRICACRYPT*, page 317–343, 2023. doi:10.1007/978-3-031-37679-5_14.

[CCF+16] A. Canteaut, S. Carpov, C. Fontaine, T. Lepoint, M. Naya-Plasencia, P. Paillier, and R. Sirdey. Stream ciphers: A practical solution for efficient homomorphic-ciphertext compression. In *Fast Software Encryption*, pages 313–333, 2016. doi:10.1007/978-3-662-52993-5_16.

[CDPP22] K. Cong, D. Das, J. Park, and H. V.L. Pereira. Sortinghat: Efficient private decision tree evaluation via homomorphic encryption and transciphering. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, page 563–577, 2022. doi:10.1145/3548606.3560702.

[CGGI16] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. TFHE: Fast fully homomorphic encryption library, August 2016. https://tfhe.github.io/tfhe/.

[CHMS22] O. Cosseron, C. Hoffmann, P. Méaux, and F.-X. Standaert. Towards case-optimized hybrid homomorphic encryption. In *ASIACRYPT 2022*, pages 32–67, 2022. doi:10.1007/978-3-031-22969-5_2.

[CIM19] S. Carpov, M. Izabachène, and V. Mollimard. New techniques for multi-value input homomorphic evaluation and applications, 2019. doi:10.1007/978-3-030-12612-4_6.

[CJP21] I. Chillotti, M. Joye, and P. Paillier. Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In *Cyber Security Cryptography and Machine Learning*, 2021. doi:10.1007/978-3-030-78086-9_1.

[CLOT21] I. Chillotti, D. Ligier, J.-B. Orfila, and S. Tap. Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for tfhe, 2021. doi:10.1007/978-3-030-92078-4_23.

[DGH+23] C. Dobraunig, L. Grassi, L. Helminger, C. Rechberger, M. Schofnegger, and R. Walch. Pasta: A case for hybrid homomorphic encryption. *IACR TCHES*, 2023. doi:10.46586/tches.v2023.i3.30-73.

[DR02] J. Daemen and V. Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard (Information Security and Cryptography)*. Springer, 2002. doi:10.1007/978-3-662-60769-5.

[DSES14] Y. Doröz, A. Shahverdi, T. Eisenbarth, and B. Sunar. Toward practical homomorphic evaluation of block ciphers using prince. In *Financial Cryptography and Data Security*, pages 208–220, 2014. doi:10.1007/978-3-662-44774-1_17.

[GBA21] A. Guimarães, E. Borin, and D. F. Aranha. Revisiting the functional bootstrap in TFHE. *IACR TCHES*, pages 229–253, 2021. doi:10.46586/tches.v2021.i2.229-253.

[GHS12] C. Gentry, S. Halevi, and N. P. Smart. Homomorphic evaluation of the aes circuit. In *CRYPTO*, pages 850–867, 2012. doi:10.1007/978-3-642-32009-5_49.

[KS25] K. Kluczniak and L. Schild. FDFB: Full domain functional bootstrapping towards practical fully homomorphic encryption. *IACR TCHES*, pages 501–537, 2025. doi:10.46586/tches.v2023.i1.501-537.

[LN14] T. Lepoint and M. Naehrig. A comparison of the homomorphic encryption schemes fv and yashe. In *AFRICACRYPT*, pages 318–335, 2014. doi:10.1007/978-3-319-06734-6_20.

[MJSC16] P. Méaux, A. Journault, F.-X. Standaert, and C. Carlet. Towards stream ciphers for efficient fhe with low-noise ciphertexts. In *EUROCRYPT*, pages 311–343, 2016. doi:10.1007/978-3-662-49890-3_13.

[MPP24] P. Méaux, J. Park, and H. V. L. Pereira. Towards practical transciphering for FHE with setup independent of the plaintext space. *IACR Communications in Cryptology*, 1(1), 2024. doi:10.62056/anxrxrxqi.

[SAP24] L. Schild, A. Abidin, and B. Preneel. Fast transciphering via batched and reconfigurable lut evaluation. *IACR Transactions on Cryptographic Hardware an d Embedded Systems*, 2024(4):205–230, 2024. doi:10.46586/tches.v2024.i4.205-230.

[SSA+07] T. Shirai, K. Shibutani, T. Akishita, S. Moriai, and T. Iwata. The 128-bit blockcipher clefia. In *Fast Software Encryption*, pages 181–195, 2007. doi:10.1007/978-3-540-74619-5_12.

[TCB+25] D. Trama, P.-E. Clet, A. Boudguiga, R. Sirdey, and N. Ye. Designing a general-purpose 8-bit (T)FHE processor abstraction. *IACR TCHES*, 2025. doi:10.46586/tches.v2025.i2.535-578.

[TCBS23] D. Trama, P.-E. Clet, A. Boudguiga, and R. Sirdey. A homomorphic aes evaluation in less than 30 seconds by means of TFHE. In *WAHC*, page 79–90, 2023. doi:10.1145/3605759.3625260.

[TKLP25] I. Thakur, A. Karmakar, C. Li, and B. Preneel. A survey on transciphering and symmetric ciphers for homomorphic encryption. Cryptology ePrint Archive, Paper 2025/093, 2025.

[WLW+24] B. Wei, X. Lu, R. Wang, K. Liu, Z. Li, and K. Wang. Thunderbird: Efficient homomorphic evaluation of symmetric ciphers in 3gpp by combining two modes of TFHE. *IACR TCHES*, page 530–573, 2024. doi:10.46586/tches.v2024.i3.530-573.

[WWL+23] B. Wei, R. Wang, Z. Li, Q. Liu, and X. Lu. Fregata: Faster homomorphic evaluation of AES via TFHE. In *ISC 2023*, pages 392–412, 2023. doi:10.1007/978-3-031-49187-0_20.

[YXS+21] Z. Yang, X. Xie, H. Shen, S. Chen, and J. Zhou. Tota: Fully homomorphic encryption with smaller parameters and stronger security. Cryptology ePrint Archive, Report 2021/1347, 2021.